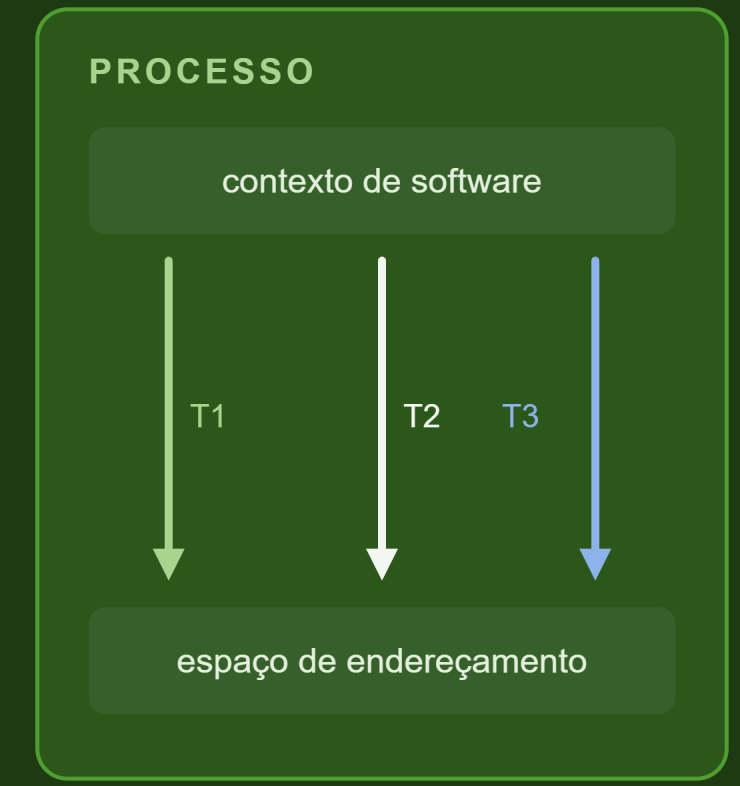
ARQUITETURA DE SISTEMAS OPERACIONAIS · CAPÍTULO 6

Thread

Vários fluxos de execução dentro de um mesmo processo: por que existem, como funcionam e como o SO os implementa

Prof. Christiano Rossini · IFAL Campus Maceió



CAP. 6 · ROTEIRO

A sequência do capítulo

6.1
Introdução

6.2
Ambiente monothread

6.3
Ambiente multithread

6.4.1
Threads em modo usuário

6.4.2
Threads em modo kernel

6.4.3
Threads em modo híbrido

6.4.4
Scheduler activations

6.5
Modelos de programação

+
Prática com Pthreads e revisão

De processos pesados a threads

até o fim dos anos 1970

Sistemas como Tops-10, MVS e Unix só têm processos com **um único thread**.

1979

No sistema Toth surge a ideia de **processos leves** (lightweight).

anos 1980

O Mach separa com clareza **processo** e **thread**.

hoje

Windows, Linux, macOS, Android: **multithread** é o padrão.

Com threads, partes diferentes do mesmo programa executam concorrentemente, com **menos custo** do que usando vários processos.

Monothread: um processo, um fluxo

PROCESSO

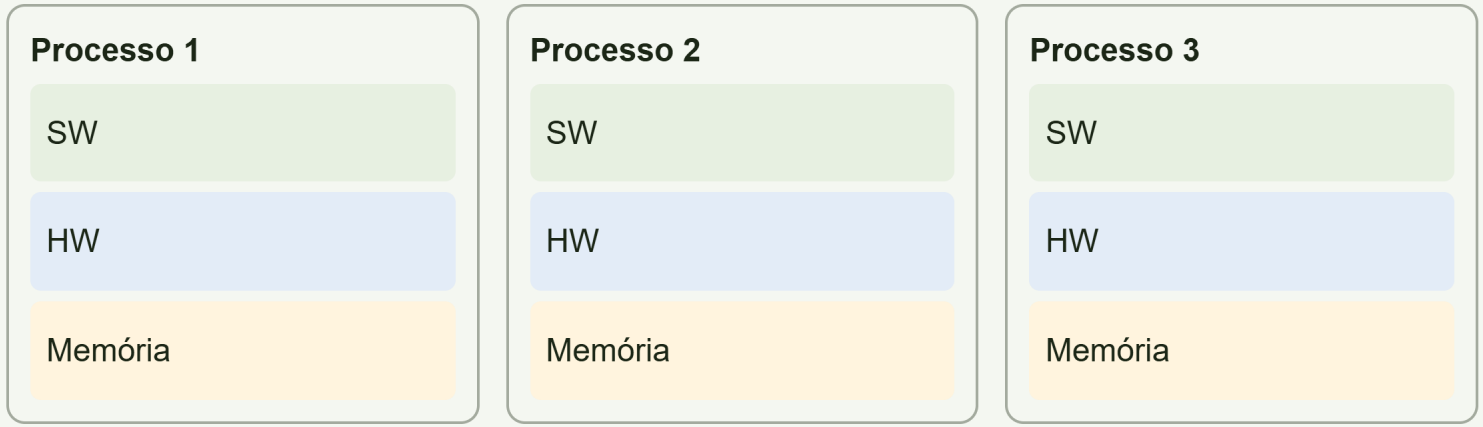
Contexto de software

Contexto de hardware

Espaço de endereçamento

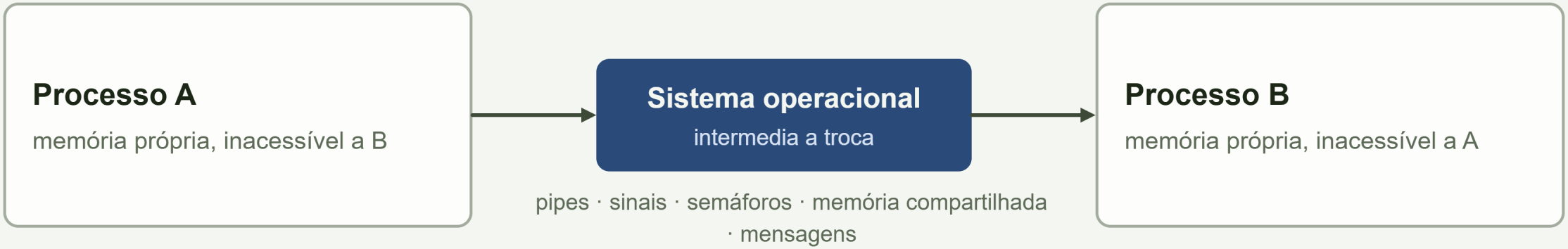
Um único programa, um único fluxo de execução.

Para ter concorrência, só criando mais processos



A aplicação é dividida em **processos independentes** ou **subprocessos**, e cada um carrega o contexto completo.

O preço da concorrência com processos



Criar e eliminar

Cada processo exige alocar e liberar PCB, memória e recursos.

Trocar

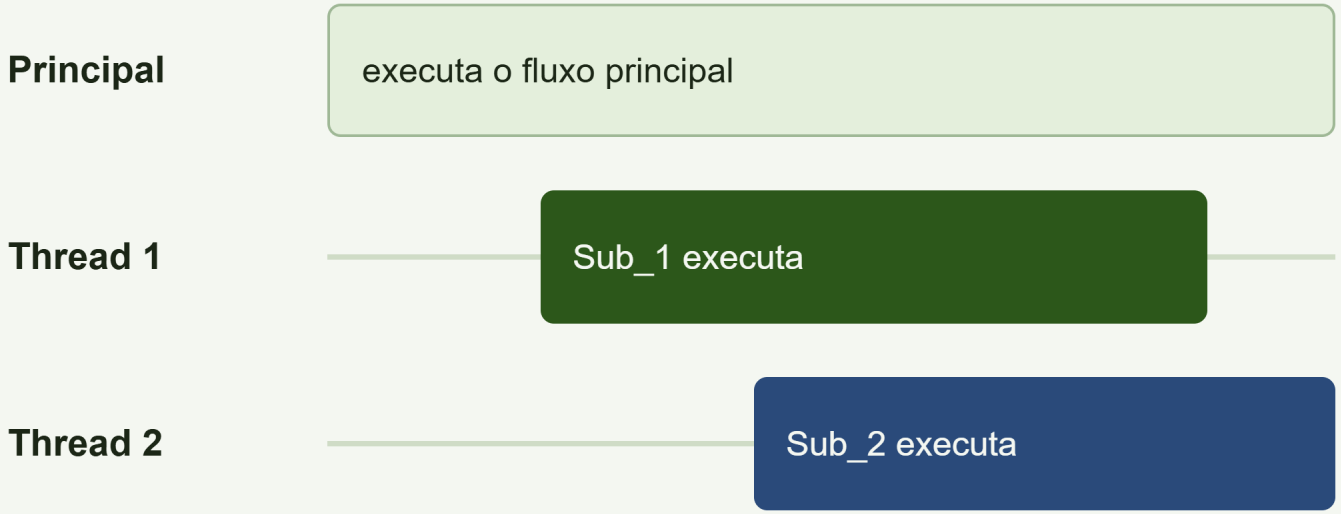
A mudança de contexto entre processos é completa e custa UCP.

Comunicar

Espaços separados: toda troca de dados passa pelo SO, o que é lento e complexo.

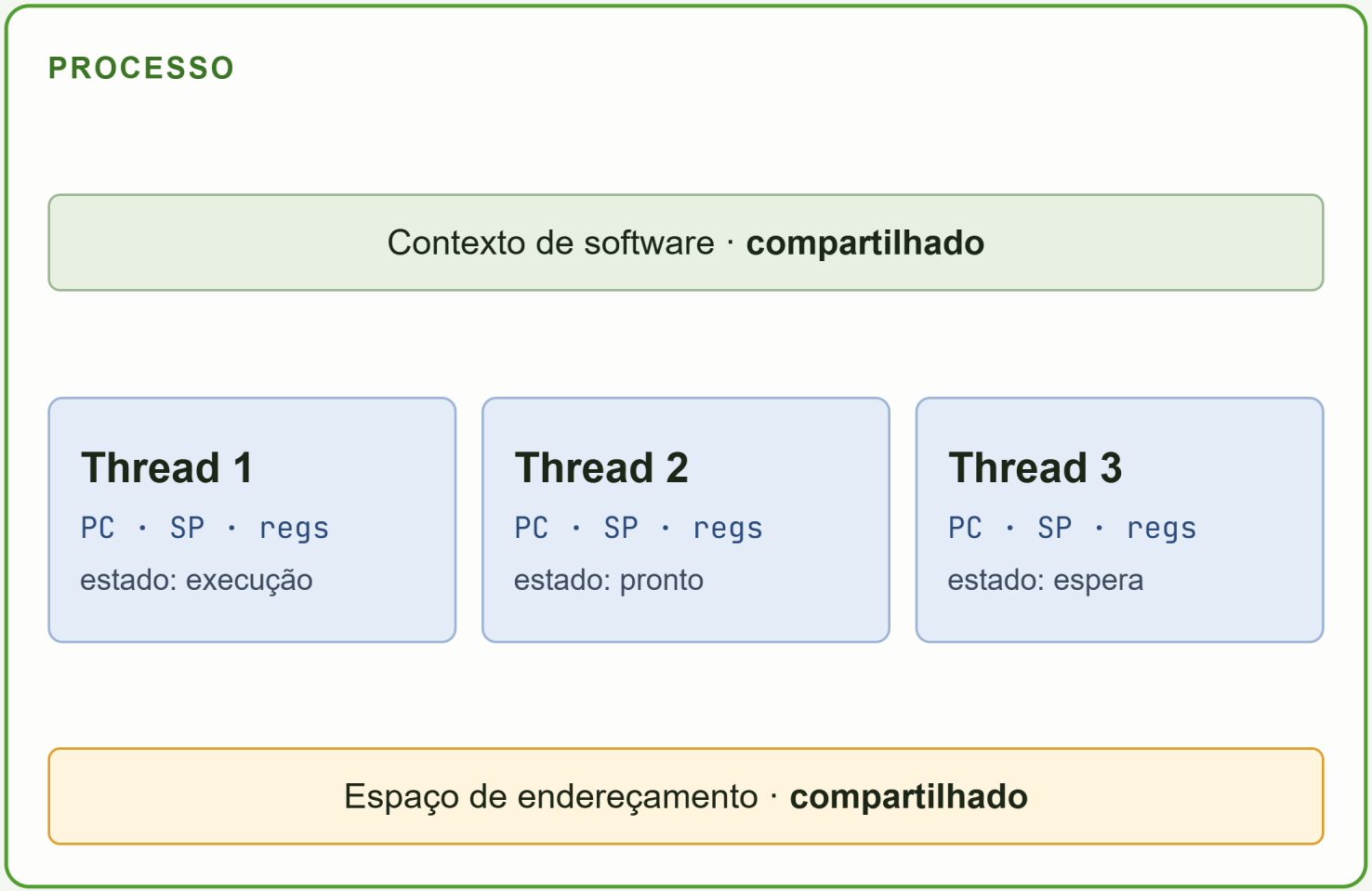
Multithread: sub-rotinas que executam juntas

```
programa principal
Create_Thread(Sub_1)
Create_Thread(Sub_2)
...continua
fim
sub-rotina Sub_1 ...
sub-rotina Sub_2 ...
```



Um **thread** é uma sub-rotina executada de forma **assíncrona**, concorrentemente com o resto do programa, dentro do mesmo processo.

O que cada thread tem e o que todos dividem



Próprio

contexto de hardware e estado de cada thread

Comum

contexto de software e memória do processo

Dois papéis que antes eram um só

PROCESSO

Unidade de alocação de recursos

Recebe memória, arquivos abertos, quotas e privilégios.

THREAD

Unidade de escalonamento

É quem recebe a UCP e passa por execução, pronto e espera.

Memória compartilhada: ganho e risco

Vantagens

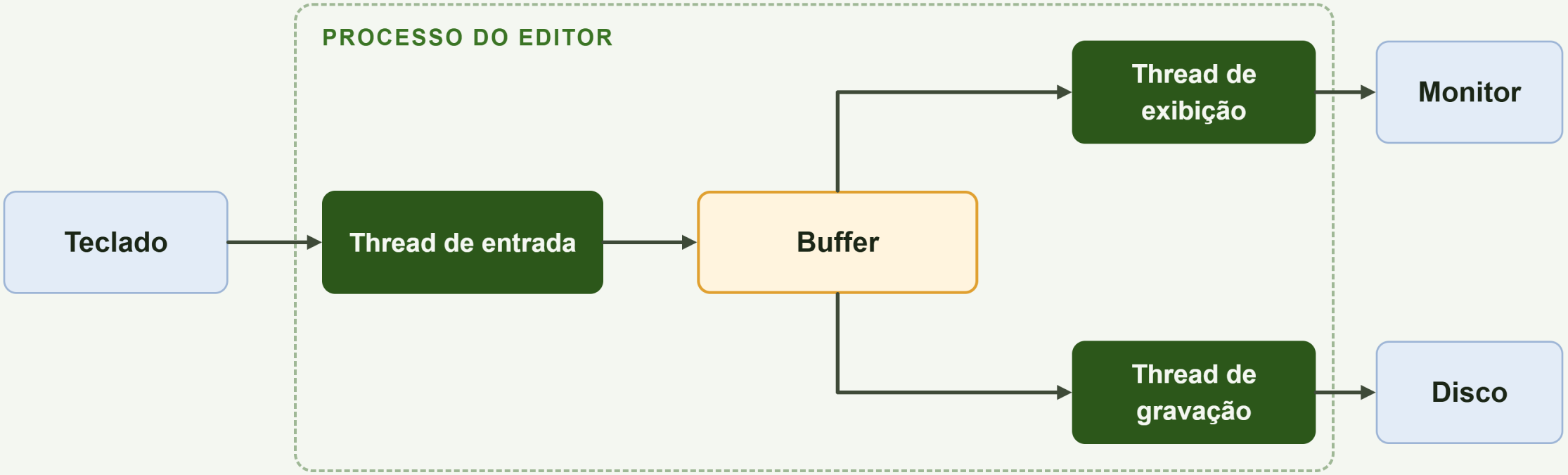
- criar e trocar threads é rápido
- comunicação por variáveis na memória, sem passar pelo SO
- menos recursos consumidos

Desvantagens

- não há proteção entre threads do mesmo processo
- um thread pode alterar ou corromper dados de outro
- exige sincronização (cap. 7)

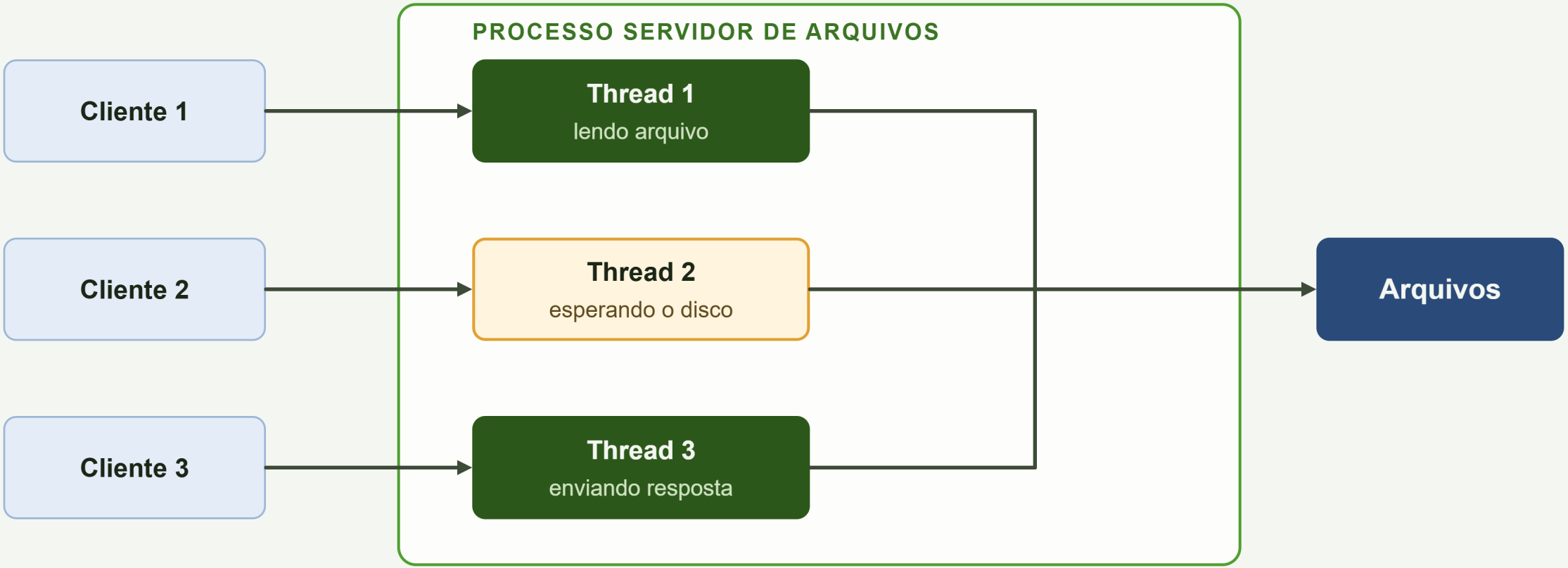
```
T1: saldo = saldo + 100 ‖ T2: saldo = saldo - 50 → resultado depende da ordem
```

Exemplo: um editor de texto multithread



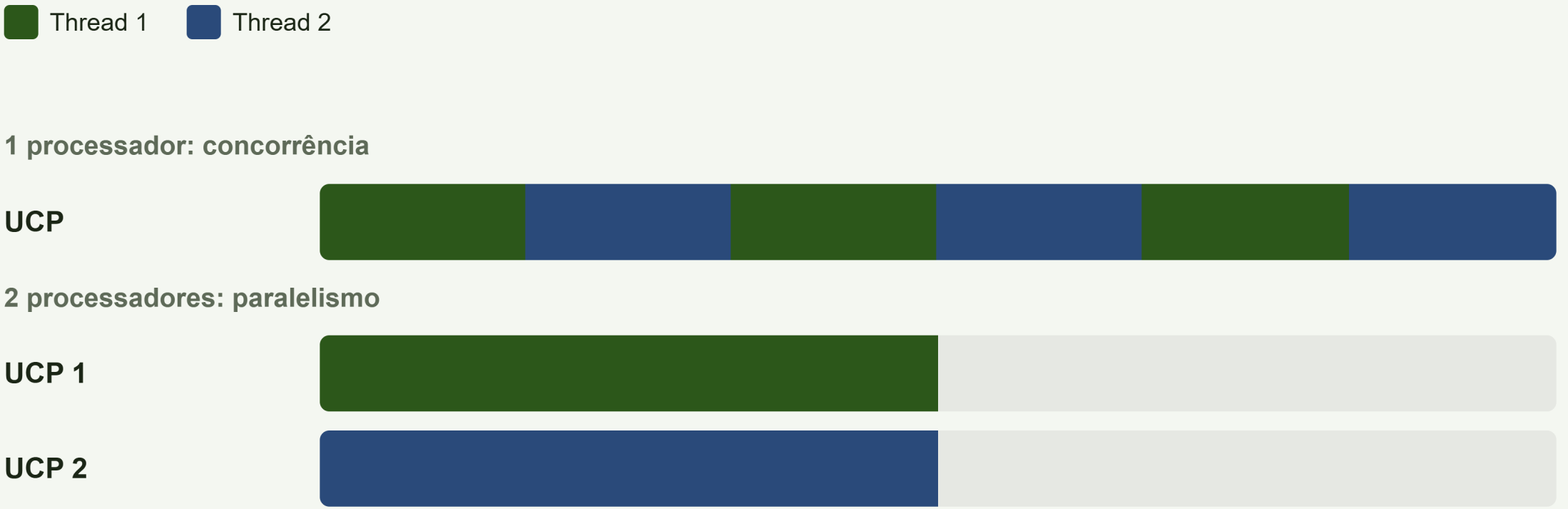
Enquanto você digita, o texto aparece na tela e é salvo em disco **ao mesmo tempo**. Nenhuma tarefa espera a outra.

Exemplo: um servidor multithread



Cada pedido é atendido por um thread: enquanto um espera o disco, os outros continuam trabalhando.

Com vários processadores, paralelismo real



Os threads de um mesmo processo executam **ao mesmo tempo** em processadores diferentes, e a aplicação termina antes.

Onde fica o pacote de threads?

O conjunto de rotinas que cria, elimina, escalona e sincroniza threads pode ficar fora do núcleo, dentro dele, ou nos dois.

6.4.1

Modo usuário

biblioteca fora do núcleo; o SO não vê os threads

N : 1

6.4.2

Modo kernel

o próprio núcleo cria e escalona cada thread

1 : 1

6.4.3

Modo híbrido

threads de usuário sobre threads de kernel

M : N

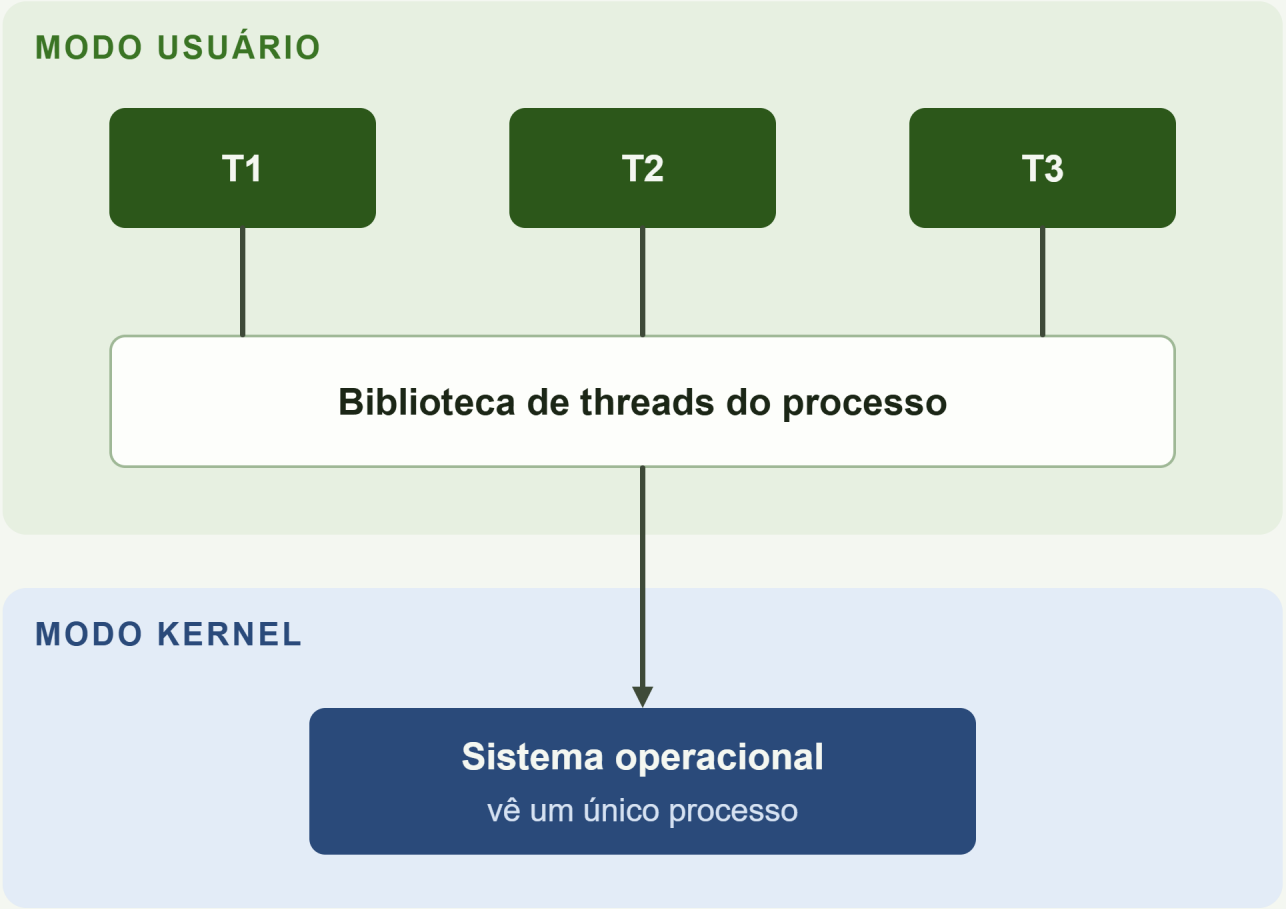
6.4.4

Scheduler activations

núcleo e biblioteca cooperam, trocando avisos

M : N +

Threads em modo usuário (TMU)



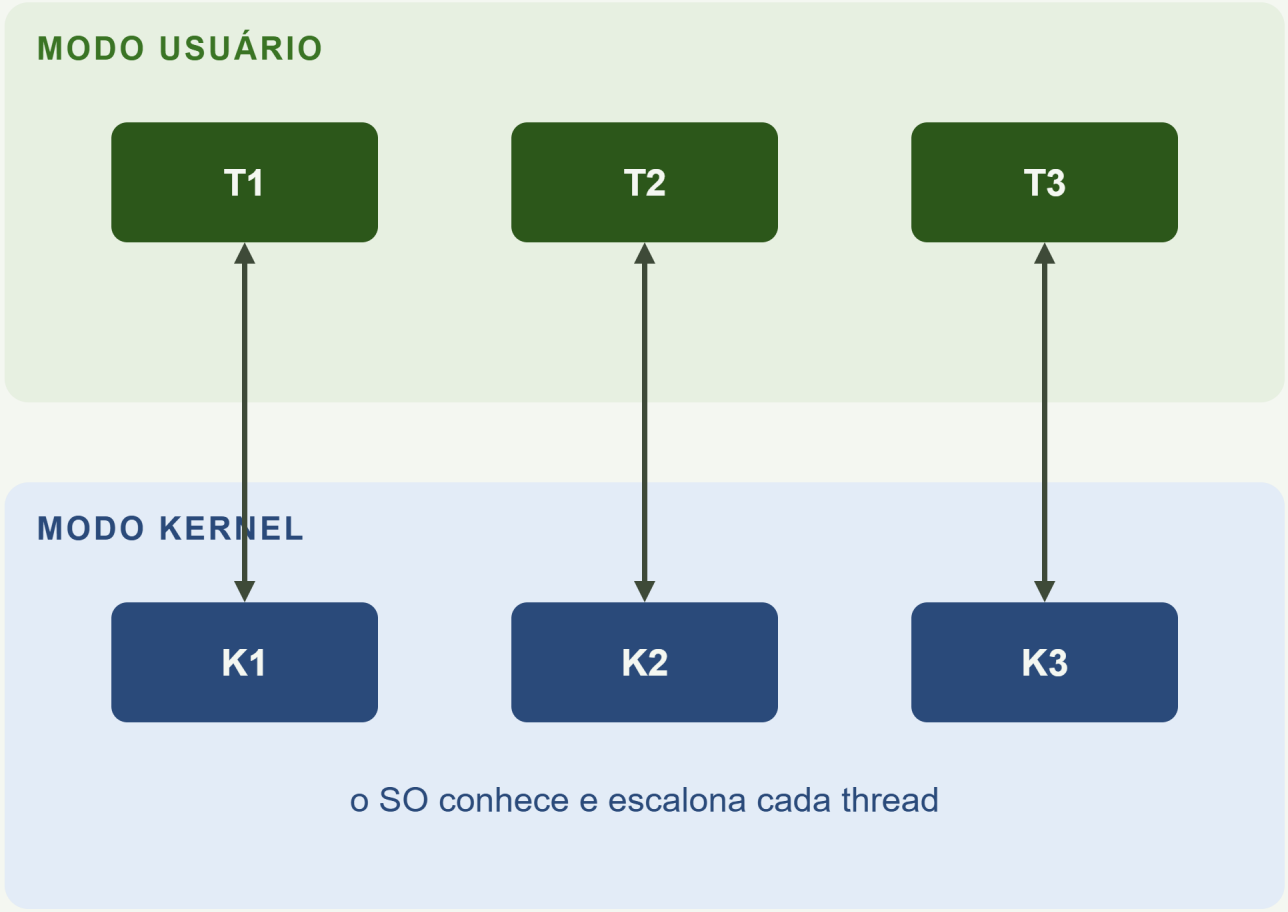
Vantagens

Troca entre threads sem chamar o SO: muito rápida. Roda até em SO sem suporte a threads.

Desvantagens

Uma chamada bloqueante (ex.: E/S) bloqueia o processo inteiro. Sem paralelismo entre processadores: o SO escalona o processo, não os threads.

Threads em modo kernel (TMK)



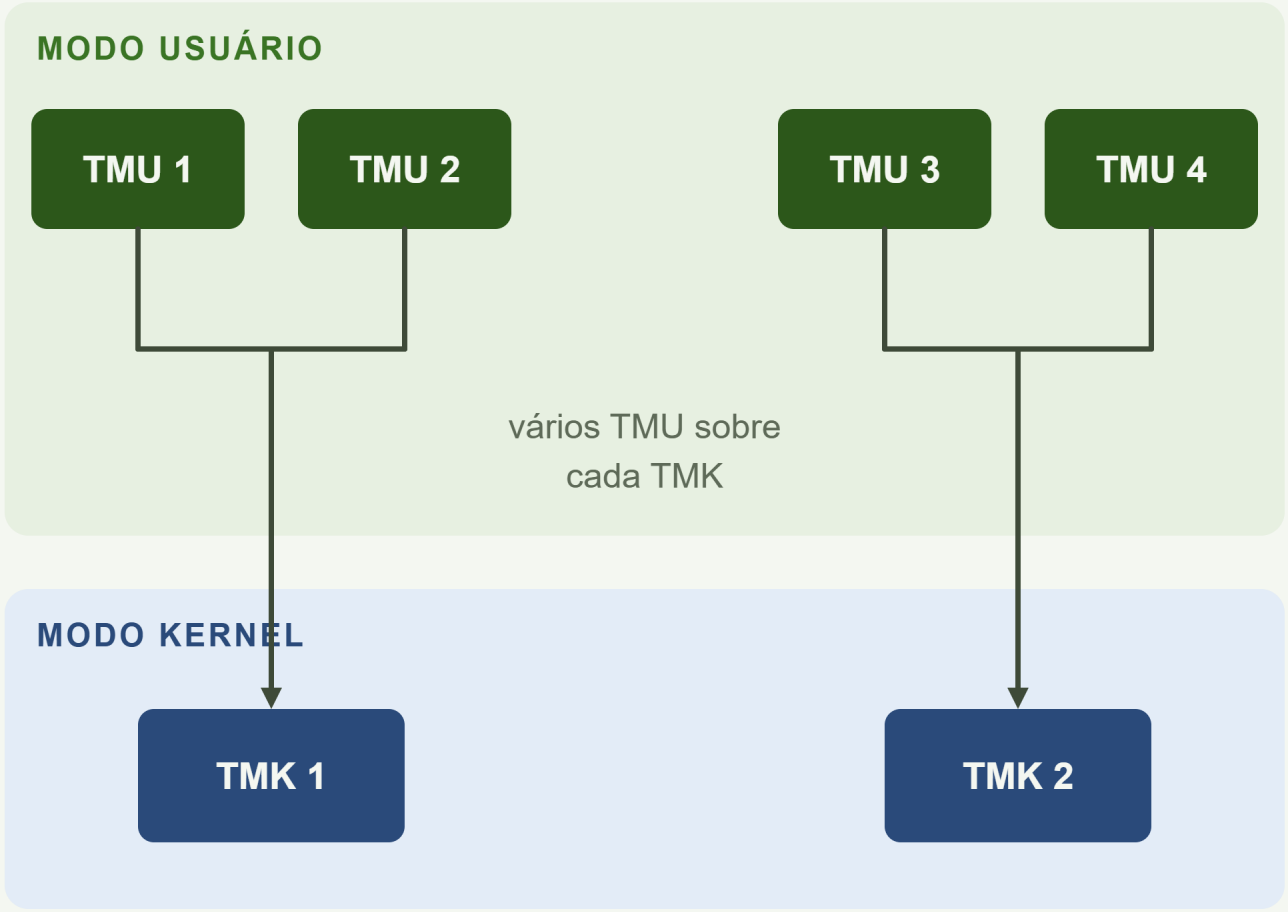
Vantagens

Um thread bloqueado não para os outros. Threads do mesmo processo rodam em paralelo em vários processadores.

Desvantagens

Criar, trocar e sincronizar threads exige chamadas ao SO e mudanças de modo: desempenho menor.

Threads em modo híbrido



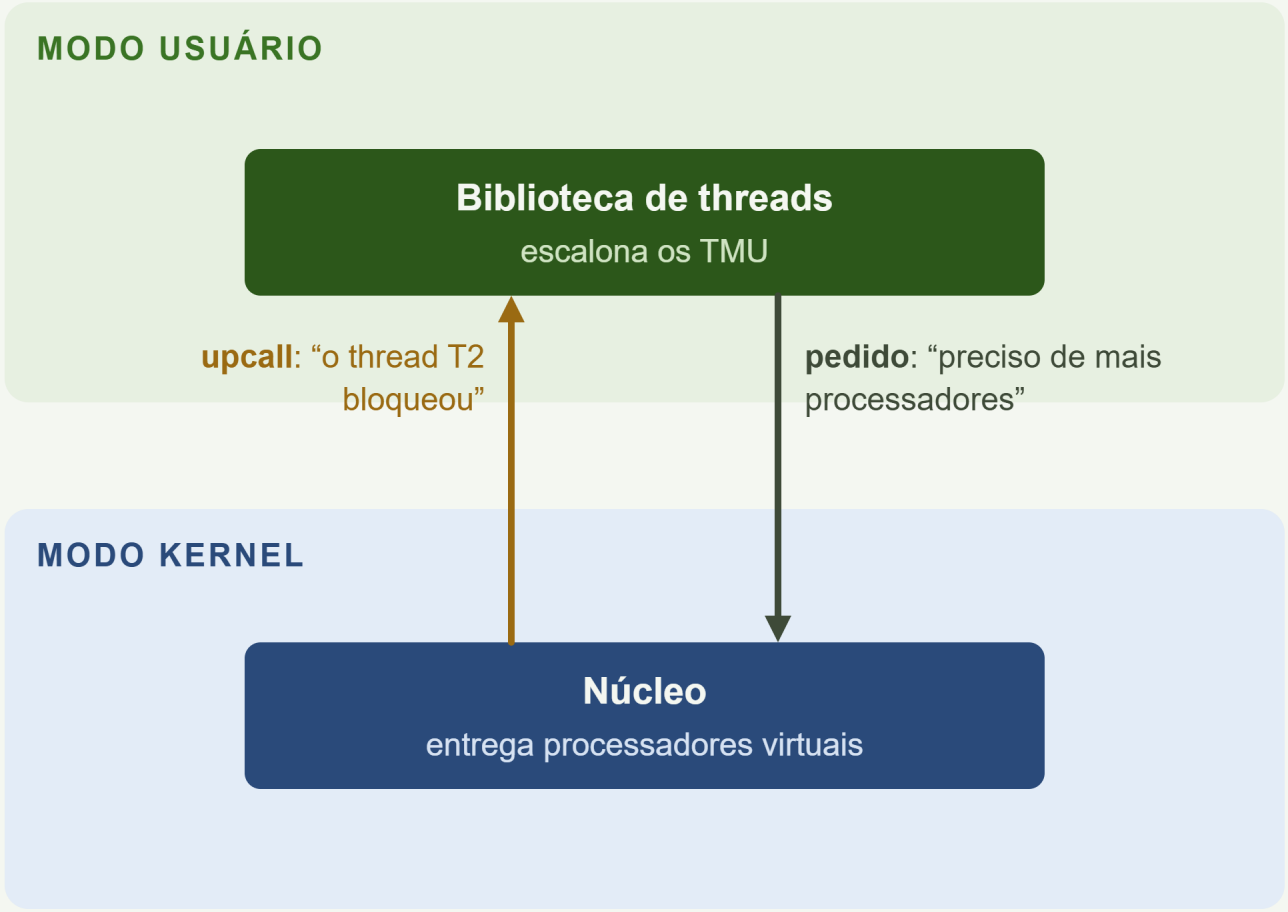
Vantagens

Flexível: junta a troca rápida dos TMU com o paralelismo dos TMK.

Desvantagens

Herda problemas dos dois: se um TMU faz chamada bloqueante, o TMK que o executa bloqueia, e com ele todos os TMU associados.

Scheduler activations: núcleo e biblioteca conversam



A ideia

O núcleo avisa a biblioteca dos eventos; a biblioteca decide qual thread rodar.

Frente ao híbrido

Quando um thread bloqueia, a biblioteca é avisada e põe outro para rodar: os demais não ficam presos.

As quatro formas lado a lado

Critério	Modo usuário	Modo kernel	Híbrido	Scheduler activations
Quem gerencia	biblioteca	núcleo	ambos	ambos, cooperando
Troca entre threads	rápida	lenta (muda de modo)	rápida entre TMU	rápida
Chamada bloqueante	bloqueia o processo	bloqueia só o thread	bloqueia o TMK e seus TMU	só o thread; a biblioteca é avisada
Paralelismo em vários processadores	não	sim	sim	sim
Mapeamento	N : 1	1 : 1	M : N	M : N

Programar com threads exige cuidado

Sincronizar

Dados compartilhados precisam de acesso coordenado, ou surgem inconsistências e deadlocks.

Decidir onde e quantos

Só há ganho se partes do código puderem executar ao mesmo tempo.

Depurar é difícil

A ordem de execução muda a cada vez: um erro pode aparecer só às vezes.

Usar um padrão

POSIX Threads (Pthreads) define uma interface portável para criar e sincronizar threads.

Três threads em C com Pthreads

```c
#include <pthread.h>
#include <stdio.h>
void *tarefa(void *arg) {
printf("thread %ld\n", (long) arg);
return NULL;
}
int main(void) {
pthread_t t[3];
for (long i = 0; i < 3; i++)
pthread_create(&t[i], NULL, tarefa, (void *) i);
for (int i = 0; i < 3; i++)
pthread_join(t[i], NULL);
return 0;
}
```

`gcc prog.c -o prog -pthread`

Rode várias vezes: a ordem das mensagens muda.

`ps -eLf`

Coluna LWP: um identificador por thread.

`top -H`

Mostra threads em vez de processos.

`ls /proc/PID/task`

Uma pasta por thread do processo.

CAP. 6 · RESUMO

O capítulo em um quadro

Thread = fluxo de execução com **contexto de hardware próprio**, que divide com os outros threads o contexto de software e a memória do processo.

Monothread

- concorrência só com processos
- caro para criar e trocar
- comunicação via SO

Multithread

- processo: alocação de recursos
- thread: escalonamento
- memória compartilhada

Implementação

- modo usuário
- modo kernel
- híbrido
- scheduler activations

Programação

- sincronização
- depuração difícil
- Pthreads

Para revisar

1. Quais os problemas de fazer concorrência só com processos?
2. Qual a diferença entre unidade de alocação de recursos e unidade de escalonamento?
3. Por que compartilhar a memória entre threads é, ao mesmo tempo, vantagem e risco?
4. O que acontece com os outros threads quando um thread de modo usuário faz uma E/S?
5. O que o scheduler activations resolve em relação ao modelo híbrido?